哈尔滨工业大学（深圳）

软件构造实验指导书

实验一 系统分析及单例模式

2026 春

目录

1. 实验目的.....3
2. 实验环境.....3
3. 实验内容.....3
4. 实验步骤.....3
4.1 飞机大战任务书.....3
4.2 模板程序导入及运行.....6
4.3 核心实体类设计与 UML 建模.....8
4.3.1 UML 类图.....8
4.3.2 类与类之间的关系.....9
4.3.3 PlantUML 插件.....11
4.4 单例模式应用.....15
4.4.1 英雄机场景分析.....15
4.4.2 代码实现.....16
5. 本次迭代开发目标.....17

1. 实验目的

1. 理解面向对象编程的基本思想和核心概念；
2. 结合具体实例，掌握面向对象分析与设计的方法；
3. 掌握 UML 类图绘制规范，能够使用类图准确表达系统的继承关系；
4. 深入理解单例模式的设计意图及 UML 结构，并能通过代码实现与应用。

2. 实验环境

1. IntelliJ IDEA 2025.3
2. OpenJDK 25

3. 实验内容

1. 理解飞机大战系统功能，导入并运行模板程序
2. 分析核心实体类的继承关系并完成 UML 建模，用 PlantUML 绘制完整类图
3. 重构代码，新增 4 种敌机、5 种道具及其抽象父类，完善继承体系
4. 理解单例模式设计意图，绘制英雄机单例模式的 UML 结构图
5. 重构英雄机创建逻辑，采用单例模式保证全局唯一实例
6. 完成本次迭代开发清单中的所有任务

4. 实验步骤

4.1 飞机大战任务书

1. 角色设定

名称：英雄机		预期应用设计模式
实例数量	1	单例模式
种类	1 种：英雄机	
移动方式	跟随玩家鼠标移动	
生存	通过生命值（血）生存，被敌机子弹击中损失部分生命值，被敌机碰撞则全部损失	
火力	自动发射子弹，如有火力道具加成则改变弹道	策略模式

消灭	生命值为 0 时，游戏结束	

名称：敌机		预期应用设计模式
实例数量	敌机最大同屏数量（enemyMaxNumber）	工厂模式
种类	5 种：普通敌机、精英敌机、精锐敌机、王牌敌机、Boss 敌机	
出现方式	Boss 敌机：玩家分数达到指定阈值时触发生成 其它敌机：每隔固定周期，在界面上方随机产生一种，且只生成一架	
移动方式	普通敌机、精英敌机：仅向屏幕下方移动	
	精锐敌机、王牌敌机：向屏幕下方移动，且可左右移动	
	Boss 敌机：悬浮于界面上方，仅左右移动	
生存	通过生命值生存，被英雄机子弹击中损失部分生命值，生命值为 0 或与英雄机碰撞时坠毁	
火力	普通敌机不发射子弹，其他敌机按照一定轨迹、频率发射子弹	策略模式
消灭	生命值为 0 时，敌机坠毁。玩家总分增加相应分数。普通敌机不掉落道具，其它敌机随机掉落某种道具	

名称：道具		预期应用设计模式
实例数量	不限	简单工厂模式
种类	5 种：火力道具、超级火力道具、炸弹道具、加血道具、冰冻道具	
出现方式	敌机坠毁时按预设概率随机掉落	
使用方式	英雄机碰撞后自动触发	
移动方式	以一定速度向屏幕下方移动	
效果	火力道具、超级火力道具：英雄机切换弹道并持续一段时间，结束后恢复原状态	

	加血道具：可使英雄机恢复一定血量，但不能超过英雄机初始的最大血量	
	炸弹道具：Boss 敌机不受影响，对王牌敌机造成伤害，清除界面上其它敌机和所有敌机子弹，英雄机可获得所有坠毁敌机的分数	观察者模式
	冰冻道具：Boss 敌机不受影响，王牌敌机减速，精锐和精英敌机短暂冻结后恢复，普通敌机永久冻结	
消灭	与英雄机碰撞或移动至界面底部则消失	

名称：子弹		预期应用设计模式
实例数量	不限	
种类	2 种：英雄机子弹（蓝色）、敌机子弹（红色）	
速度	子弹飞行速度	
角度	子弹初始飞行方向	
移动方式	以一定速度向某个方向匀速直线运动	
消灭	击中英雄机、敌机，或飞出界面外自动消亡	

名称：排行榜		预期应用设计模式
排名方式	各游戏难度设立独立排行榜，按总分从高到低进行排名	数据访问对象模式
记录信息	游戏难度、名次、玩家名、得分、记录时间	
更新方式	系统实时累计英雄机得分并在界面左上角呈现。对局结束后，界面将展示当局分数及当前难度的历史排行榜，并引导玩家选择是否保存记录。若确认保存，玩家需录入姓名等信息以更新榜单；此外，系统还提供历史记录的删除与管理功能。	

2. 游戏难度设定

（1） 游戏设置 3 种难度：简单 / 普通 / 困难。

（2） 每种难度拥有独立的初始参数，包括：屏幕最大敌机数量、各类敌机生成概率、Boss 机触发阈值，敌机生成周期，以及敌机与英雄机的射击周期。

（3） 普通及困难模式下，游戏引入随时间递增的难度曲线。随着对局时长增加，将逐步提升敌机的血量与速度，同时延长英雄机射击周期并缩短敌机的生成与射击周期。

（4） 普通和困难模式下，玩家得分每突破一个阈值即召唤一架 Boss 机；困难模式中，每次触发的 Boss 机将拥有递增的血量上限。

3. 界面和音效需求

（1） **游戏界面：**风格清新，简单易操作。

（2） **音效：**

- ✧ 游戏背景音乐
- ✧ Boss 机背景音乐
- ✧ 敌机被子弹击中
- ✧ 道具生效
- ✧ 炸弹爆炸

游戏中循环播放游戏背景音乐，游戏结束后停止播放。子弹击中敌机、炸弹爆炸、道具生效、游戏结束时有相应的音效。Boss 敌机出场时循环播放其背景音乐，坠毁后或游戏结束后停止播放。

4.2 模板程序导入及运行

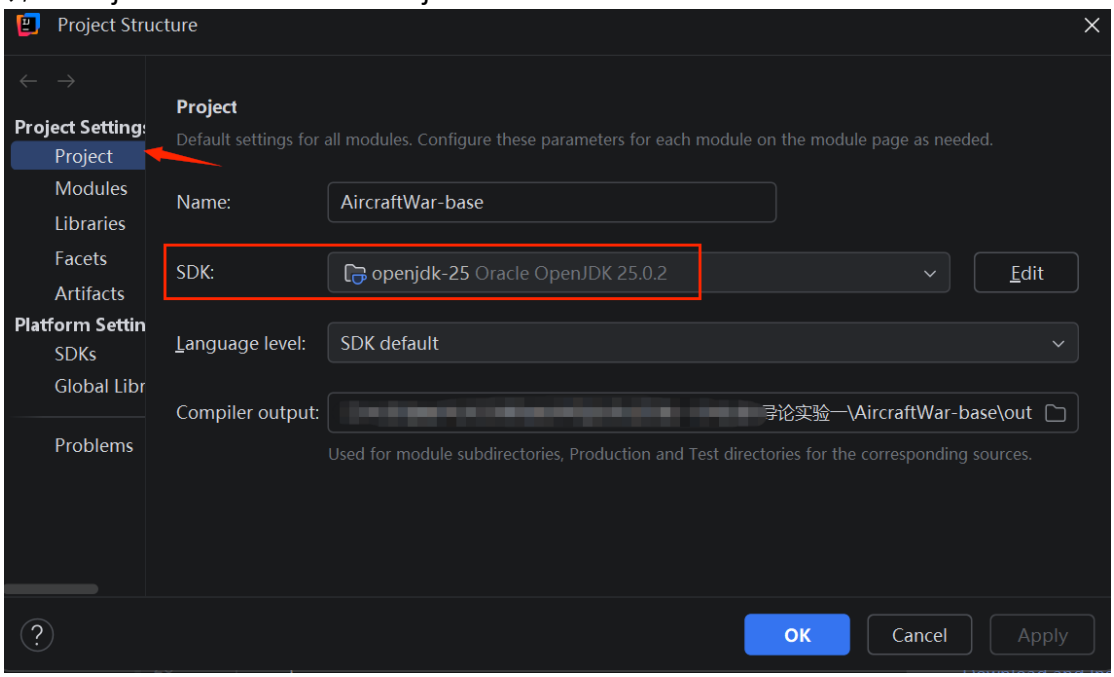
本实验提供了飞机大战的模板代码 AircraftWar-base，在模板代码中已实现如下功能：

- ✓ 游戏主界面
- ✓ 英雄机、普通敌机移动
- ✓ 英雄机子弹发射
- ✓ 碰撞检测
- ✓ 统计并显示得分和英雄机生命值

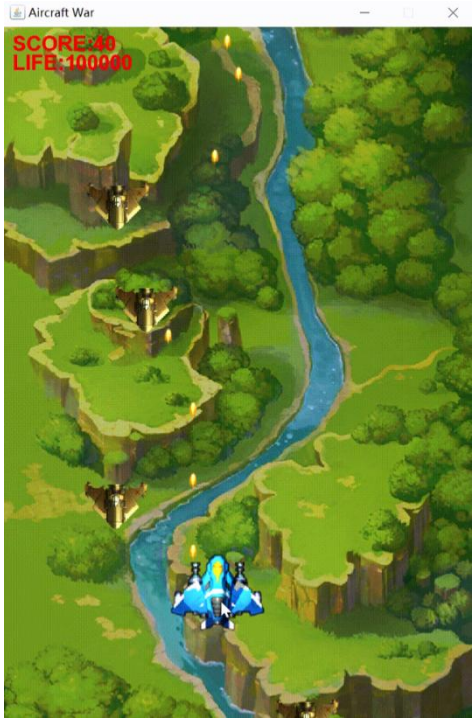
请从官网下载 Java 集成开发环境 IntelliJ IDEA（2025.3 以上）并安装。**下载**

地址：Download IntelliJ IDEA

安装后可打开模板项目 AircraftWar-base，设置项目 SDK 版本：点击“File”-> 选择“Project Structure”-> “Project”



模板代码已完成基本运行框架，并已创建英雄机和普通敌机。
可运行 src\edu\hitsz\application\Main.java 启动程序：



4.3 核心实体类设计与 UML 建模

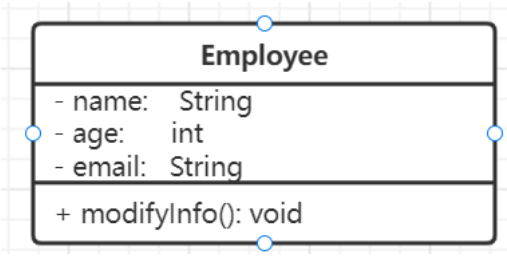
本系统包含四类核心实体，具体分类如下：
英雄机（1 种）
敌机类（5 种）：普通敌机、精英敌机、精锐敌机、王牌敌机、Boss 敌机；
道具类（5 种）：加血道具、火力道具、超级火力道具、冰冻道具、炸弹道具；
子弹类（2 种）：英雄机子弹、敌机子弹。

请**依据面向对象设计原则（如封装、继承、多态等）**，分析并设计上述所有类的层次结构。需合理提取公共特征以构建抽象父类，并派生出对应的具体子类。基于上述设计，编辑模板程序 uml 文件夹下的 **Inheritence.puml** 文件（使用 PlantUML 插件），绘制出包含所有类及其继承关系的完整 UML 类图。

要求：类图必须完整展示所有具体类、抽象父类以及它们之间的继承关系，清晰体现系统的层次结构。

4.3.1 UML 类图

类图（Class Diagram）：类图是面向对象系统建模中最常用和最重要的图，是定义其它图的基础。类图主要是用来显示系统中的类、接口以及它们之间的静态结构和关系的一种静态模型。在 UML 中，类使用包含类名、类的属性和操作且带有分隔线的长方形来表示，如定义一个 Employee 类，它包含属性 name、age 和 email，以及操作 modifyInfo()。



对应的 Java 代码片段如下：

```
public class Employee {
    private String name;
    private int age;
    private String email;

    public void modifyInfo() {
        ......
    }
}
```

4.3.2 类与类之间的关系

在画类图的时候，理清类和类之间的关系是重点。类的关系有关联(Association)、泛化(Generalization)、实现（Realization）和依赖(Dependency)。其中关联又分为一般关联关系和聚合关系(Aggregation)，组合关系(Composition)。

- **关联（Association）**

它是类与类之间最常用的一种关系，它是一种结构化关系，用于表示一类对象与另一类对象之间有联系。

① **一般关联**

【含义】:是一种拥有的关系，它使一个类知道另一个类的属性和方法；可以是双向的，也可以是单向的。如老师和学生、顾客和商品等。

【代码体现】:成员变量

【箭头及指向】:带普通箭头的实心线，指向被拥有者。双向的关联可以有两个箭头或者没有箭头，单向的关联有一个箭头。

② **聚合（Aggregation）**

【含义】:是整体与部分的关系，且部分可以离开整体而单独存在。如汽车和轮胎是整体和部分的关系，轮胎离开车仍然可以存在。聚合关系是关联关系的一种，是强的关联关系；关联和聚合在语法上无法区分，必须考察具体的逻辑关系。

【代码体现】:成员变量

【箭头及指向】:带空心菱形的实心线，菱形指向整体。

③ **组合（Composition）**

【含义】:是整体与部分的关系，但部分不能离开整体而单独存在。如公司和部门是整体和部分的关系，没有公司就不存在部门。组合关系是关联关系的一种，是比聚合关系还要强的关系，它要求普通的聚合关系中代表整体的对象负责代表部分的对象的生命周期。

【代码体现】:成员变量

【箭头及指向】:带实心菱形的实线，菱形指向整体

- **泛化（Generalization）**

【含义】:也就是继承关系，用于描述父类与子类之间的关系，父类又称基

类或超类，子类又称作派生类。表示一般与特殊的关系，它指定了子类如何特化父类的所有待征和行为。例如:老虎是动物的一种，即有老虎的特性也有动物的共性。

【箭头指向】:带三角箭头的实线，箭头指向父类

- **实现（Realization）**

【含义】:是一种类与接口的关系，表示类是接口所有特征和行为的实现者。例如：定义了一个交通工具接口 Vehicle，包含一个抽象操作 move()，在类 Ship 和类 Car 中都实现了该 move()操作，不过具体的实现细节将会不一样。Ship 和 Car 类与接口 Vehicle 是实现关系。

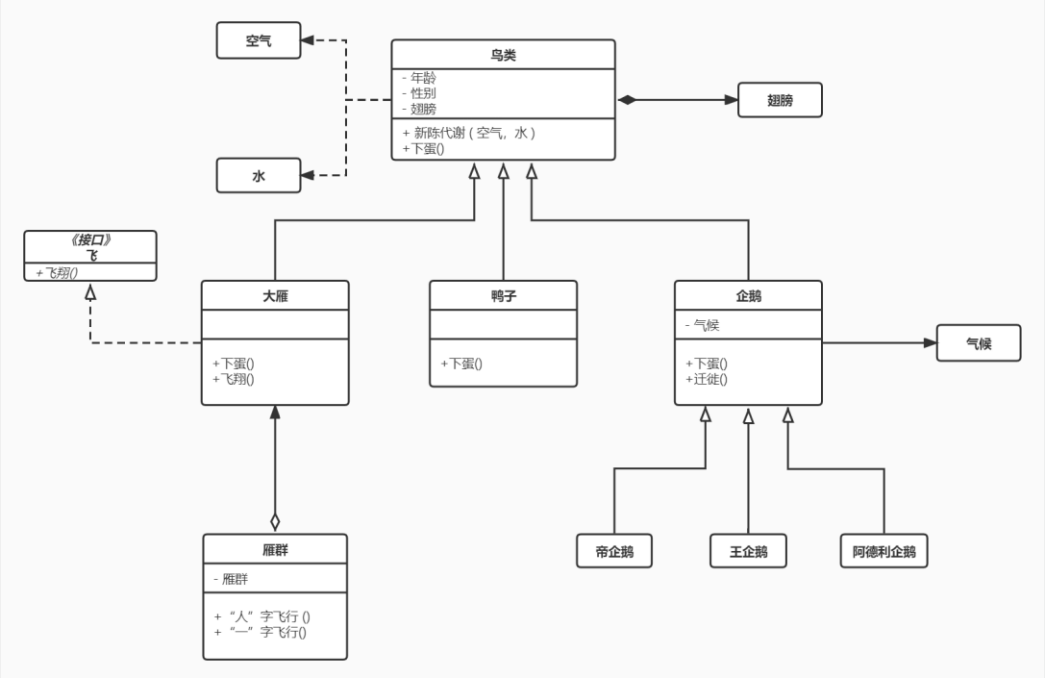
【箭头指向】:带三角箭头的虚线，箭头指向接口

- **依赖（Dependency）**

【含义】:是一种使用关系，特定事物的改变有可能会影响到使用该事物的其他事物，在需要表示一个事物使用另一个事物时使用依赖关系。例如：驾驶员开车，在 Driver 类的 drive()方法中将 Car 类型的对象 car 作为一个参数传递，以便在 drive()方法中能够调用 car 的 move()方法，且驾驶员的 drive()方法依赖车的 move()方法，因此类 Driver 依赖类 Car。

【代码表现】:局部变量、方法的参数或者对静态方法的调用

【箭头及指向】:带箭头的虚线，指向被使用者



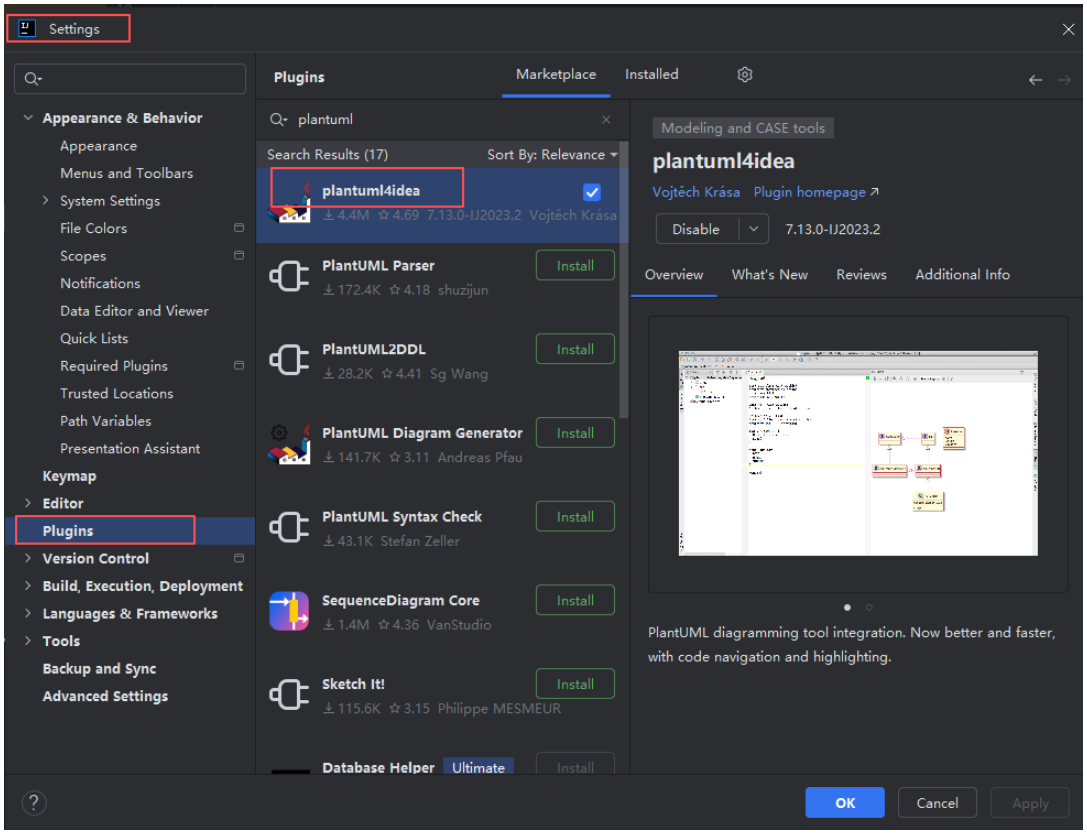
4.3.3 PlantUML 插件

PlantUML 是一个开源项目，支持快速绘制时序图、用例图、类图、对象图、活动图、组件图、部署图、状态图和定时图。Plantuml 本质上是也算一门可以快速画图的设计语言,通过简单的描述语言绘制，非常符合程序员的习惯。

(1) Intellij IDEA 安装插件

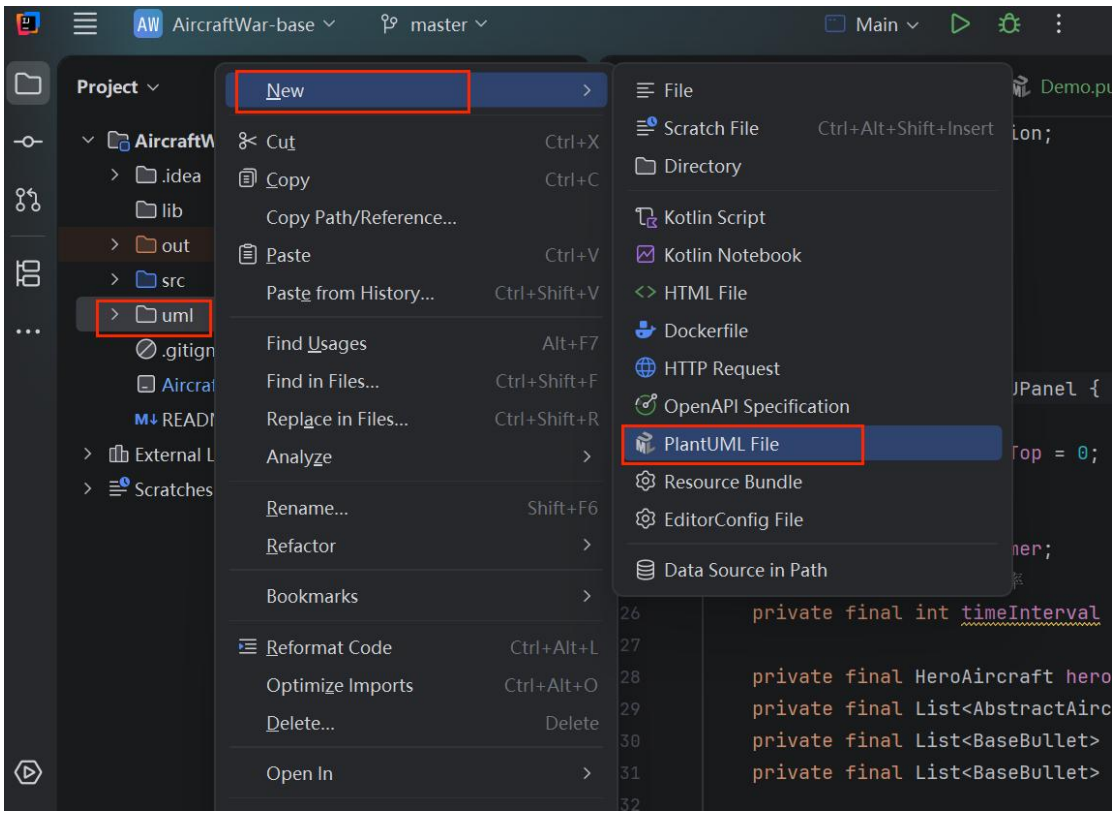
Plantuml 提供了 Intellij IDEA 插件，通过 IDEA 的插件管理搜索 PlantUml 并集成安装后，重启 IDEA 即可。

点击 File --> Settings --> Plugins --> Marketplace，搜索 PlantUml。



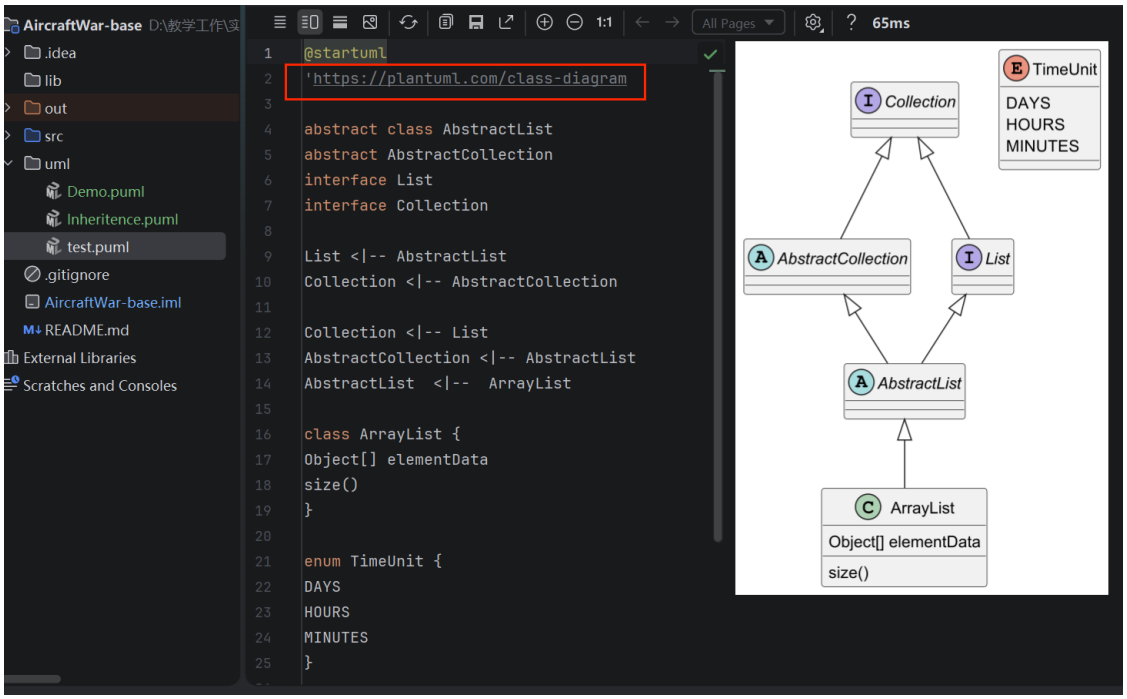
(2) PlantUML 绘制类图示例

在项目下新建 uml 文件夹，右键选择 New-->PlantUML File 创建文件，选择 Class 类图。puml 文件可以像代码一样在 IDEA 中进行管理，即通过 IDEA 可以完成日常设计，开发过程中的大多数工作。

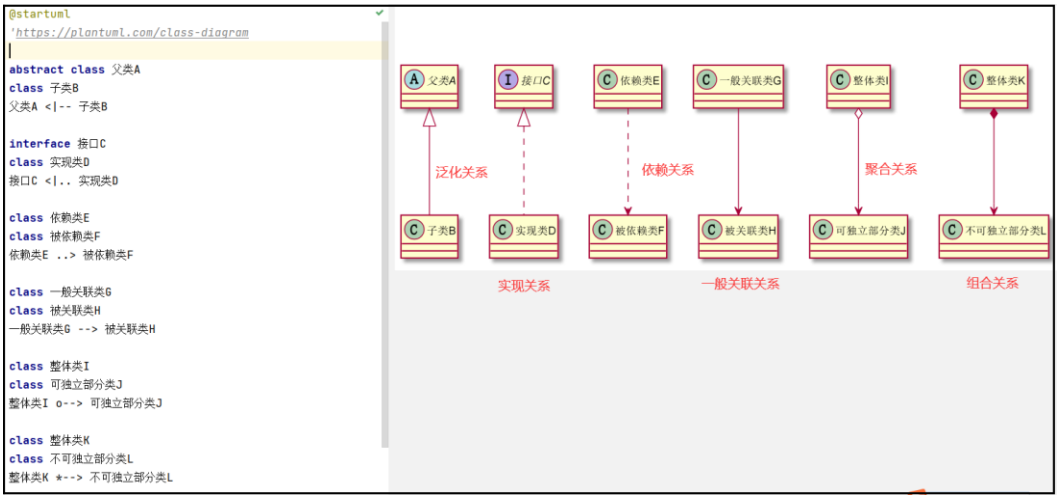


(3) plantUML 类图的语法和功能

请参考官网说明：类图的语法和功能（plantuml.com）



请注意 UML 类图绘制的规范性，包括类、抽象类、成员变量、方法、可见性、返回值、联系的符号等，严格按照指导书 4.3.1 和 4.3.2 节的要求绘制 UML 类图。



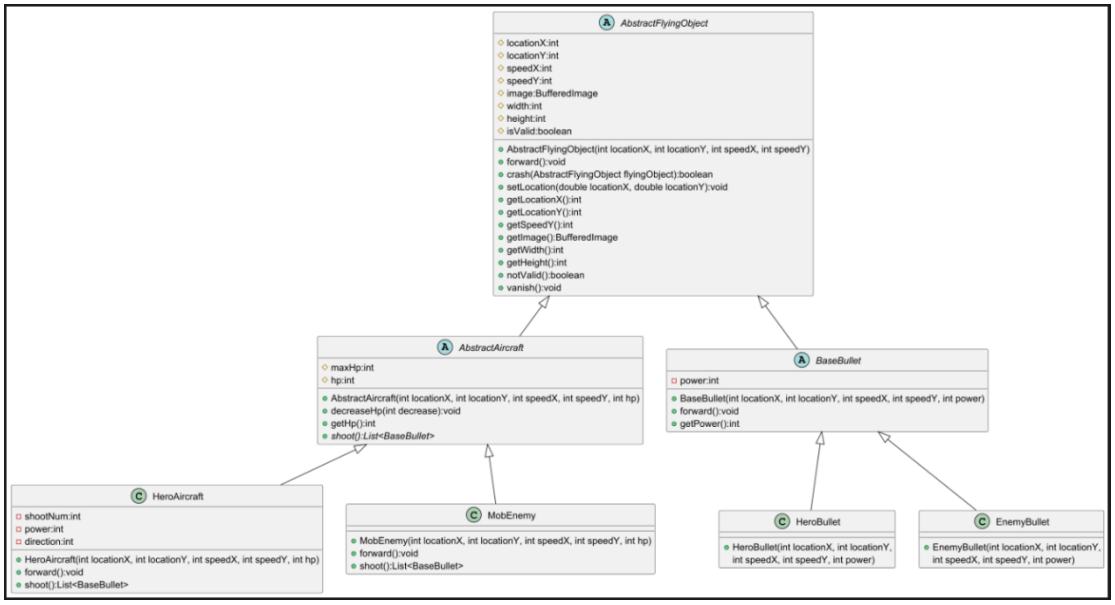
(4) 飞机大战模板程序类图示例

Inheritence.puml

```
@startuml
'https://plantuml.com/class-diagram

abstract class AbstractFlyingObject
{
        # locationX:int
        # locationY:int
        # speedX:int
        # speedY:int
        # image:BufferedImage
        # width:int
        # height:int
        # isValid:boolean

        + AbstractFlyingObject(int locationX, int locationY, int speedX, int speedY)
```

```
        + forward():void
        + crash(AbstractFlyingObject flyingObject):boolean
        + setLocation(double locationX, double locationY):void
        + getLocationX():int
        + getLocationY():int
        + getSpeedY():int
        + getImage():BufferedImage
        + getWidth():int
        + getHeight():int
        + notValid():boolean
        + vanish():void
}
abstract class AbstractAircraft
{
        # maxHp:int
        # hp:int
        + AbstractAircraft(int locationX, int locationY, int speedX, int speedY, int hp)
        + decreaseHp(int decrease):void
        + getHp():int
        + {abstract} shoot():List<BaseBullet>
  }


class HeroAircraft {
        - shootNum:int
        - power:int
        - direction:int
        + HeroAircraft(int locationX, int locationY, int speedX, int speedY, int hp)
        + forward():void
        + shoot():List<BaseBullet>
}

AbstractAircraft <|-- HeroAircraft

class MobEnemy {
        + MobEnemy(int locationX, int locationY, int speedX, int speedY, int hp)
        + forward():void
        + shoot():List<BaseBullet>
}
AbstractAircraft <|-- MobEnemy

abstract class BaseBullet
{
        - power:int
        + BaseBullet(int locationX, int locationY, int speedX, int speedY, int power)
        + forward():void
      + getPower():int
}

class HeroBullet {
        + HeroBullet(int locationX, int locationY,
          int speedX, int speedY, int power)
}
```

```
class EnemyBullet {
        + EnemyBullet(int locationX, int locationY,
          int speedX, int speedY, int power)
}

BaseBullet <|-- HeroBullet
BaseBullet <|-- EnemyBullet

AbstractFlyingObject <|-- AbstractAircraft
AbstractFlyingObject <|-- BaseBullet

@enduml
```



4.4 单例模式应用

4.4.1 英雄机场景分析

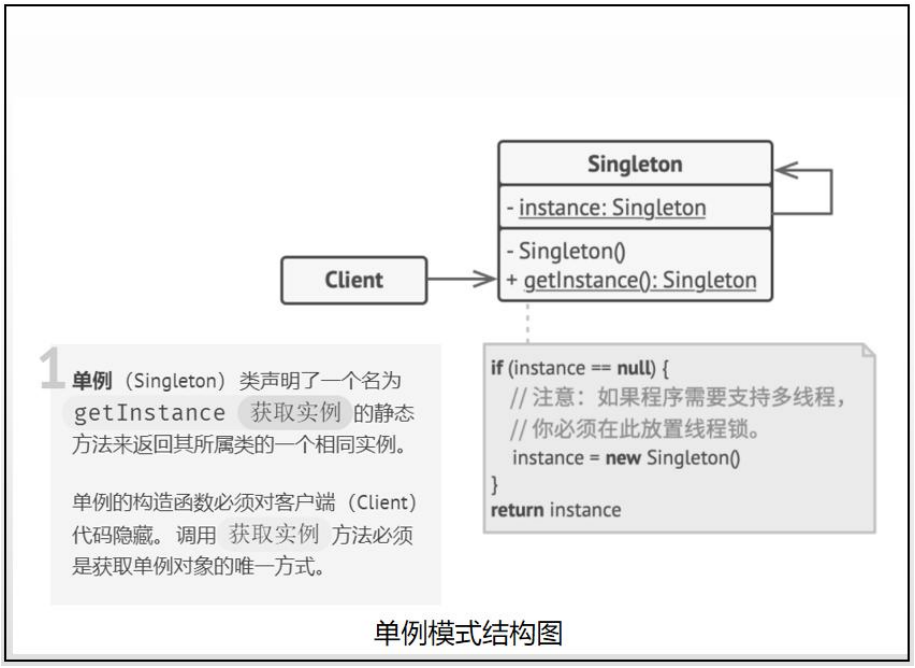
在飞机大战游戏中只有一种英雄机，且每局游戏只有一架英雄机，由玩家通过鼠标控制移动。英雄机通过生命值（血）生存，被敌机子弹击中损失部分生命值，被敌机碰撞则全部损失。英雄机生命值为 0 时判定游戏失败。

设计任务：
鉴于英雄机的“全局唯一”特性，请采用单例模式进行类设计。

UML 建模：绘制该模式的 UML 类图，图中需明确标注设计模式名称，并自定义合理的类名（HeroAircraft）、属性名及方法名，以完整展示单例模式的实现结构。

单例模式

单例模式（Singleton Pattern）是一种创建型设计模式，让你能够保证一个类只有一个实例，并提供一个访问该实例的全局节点。



单例模式结构图

请参考以上 UML 结构图，绘制飞机大战中的单例模式。

4.4.2 代码实现

依据 4.4.1 节中你所设计的 UML 类图，对现有游戏代码进行重构，采用单例模式实现英雄机类，确保全局仅存在一个英雄机实例。

参考代码示例（线程安全）：

(1) 饿汉式：类加载时即创建实例，简单但可能浪费资源

```
public class EagerSingleton {
    private static EagerSingleton instance = new EagerSingleton ();
    private EagerSingleton (){}
    public static EagerSingleton getInstance() {
        return instance;
    }
}
```

(2) 懒汉式：首次调用时创建，线程安全但性能略低

```
public class LazySingleton {
    private static LazySingleton instance = null;
    private LazySingleton (){}
    public static synchronized LazySingleton getInstance() {
    if (instance == null) {
        instance = new LazySingleton();
    }
    return instance;
    }
}
```

(3) 双重检查锁定（DCL，即 double-checked locking）：兼顾延迟加载与高性能，推荐使用

```
public class Singleton {
    private volatile static Singleton singleton;
    private Singleton (){}
    public static Singleton getSingleton() {
        if (singleton == null) {
            synchronized (Singleton.class) {
                if (singleton == null) {
                    singleton = new Singleton();
                }
            }
        }
        return singleton;
    }
}
```

5. 本次迭代开发目标

本阶段为系统原型构建期，重点在于搭建核心类层次结构与验证基础交互逻辑。本次迭代仅需完成下列清单任务，实现框架或模拟输出，任务书中其余细节功能将在后续迭代中逐步完善。

本次迭代任务清单（Checklist）：

（1）**UML 类图绘制：**完善继承关系类图、绘制单例模式结构图

（2）**设计模式应用：**重构代码，采用单例模式创建英雄机

（3）**核心实体类扩展：**在项目中增加其余 4 种敌机、5 种道具以及它们的抽象

父类,完善继承结构。（注：仅需完成类定义，无需实现复杂逻辑和功能）

（4）**敌机生成：**界面上可观察到普通敌机和精英敌机按周期随机出现

（5）**敌机攻击：**精英敌机按设定周期向下直射单排子弹

（6）**道具生成：**精英敌机坠毁时，以一定概率随机生成一个道具（范围限定为：加血道具、火力道具、超级火力道具）

（7）**道具生效：**英雄机拾取道具后，道具自动生效并消失

（8）**道具效果：**加血道具可使英雄机恢复一定血量，但不超过初始最大血量；两种火力道具无需具体实现，生效时只需在控制台打印“FireSupply active!”“FirePlusSupply active!”语句即可。